BE-03: Technology Selection

Intelligent Kanban System с AI-ассистентом
для управления задачами голосом и текстом

Команда проекта

Версия 3 · 20 октября 2026

Изменения относительно версии 2: добавлены frontend, распознавание намерений для 8 AI-действий, права доступа, уведомления, хранение и защита голосовых сообщений, аудит. Решение «локально или облако» пересмотрено с точки зрения реального продукта: скорость, масштабируемость, качество казахского языка.

Содержание

1 Цель **3**

2 Требования, определяющие выбор **3**

3 Архитектура **4**

3.1 AI-конвейер 4

4 Speech-to-Text **5**

4.1 Сравнение 5

4.2 Решение: self-hosted STT с маршрутизацией по языку и облачный fallback 5

5 LLM для распознавания команд **6**

5.1 Что должна уметь модель 6

5.2 Сравнение 6

5.3 Решение: гибрид через единый интерфейс `CommandParser` 6

5.4 Защита от выдуманных данных 7

6 Frontend **7**

7 Backend **8**

8 База данных и хранилища **8**

9 Безопасность и защита данных **8**

10 Уведомления **9**

11 Итоговый стек **9**

12 Проверка выбора **10**

13 Риски **10**

14 Источники **11**

1. Цель

Выбрать и обосновать технологии для **коммерческого Kanban-продукта** для сотрудников компаний, руководителей проектов, преподавателей и студентов. AI-ассистент встроен в рабочий процесс: пользователь говорит или пишет команду, система её понимает, показывает предлагаемое действие и после подтверждения выполняет его на доске.

2. Требования, определяющие выбор

ID	Требование	Влияние на стек
R1	Языки kk, ru, en и смешанная речь (kk + ru + английские термины)	Качество STT и LLM на казахском — главный критерий
R2	8 AI-действий: Create Task, Update Task, Change Status, Assign User, Change Deadline, Change Priority, Search Tasks, Project Summary	LLM с надёжным structured output / tool calling
R3	AI не меняет данные без подтверждения и при неоднозначности; не выдумывает исполнителей, сроки и задачи	Детерминированная проверка на backend, AI только предлагает
R4	Полноценный продукт: auth, workspaces, проекты, участники, задачи, комментарии, история, уведомления	Реляционная БД, RBAC, real-time обновления
R5	Ответ на голосовую команду за несколько секунд	Быстрый STT, асинхронная обработка
R6	Защита пользовательских данных и голосовых	Шифрование, ограниченный срок хранения, контроль доступа
R7	Масштабируемость и развитие продукта	Stateless API, отдельный AI-воркер, замена моделей конфигурацией
R8	Надёжность: ошибки AI, недоступность API, повторные запросы	Fallback-провайдеры, идемпотентность, очереди
R9	Цифровой след разработки (GitLab, YouTrack)	GitLab CI, тесты, история

3. Архитектура

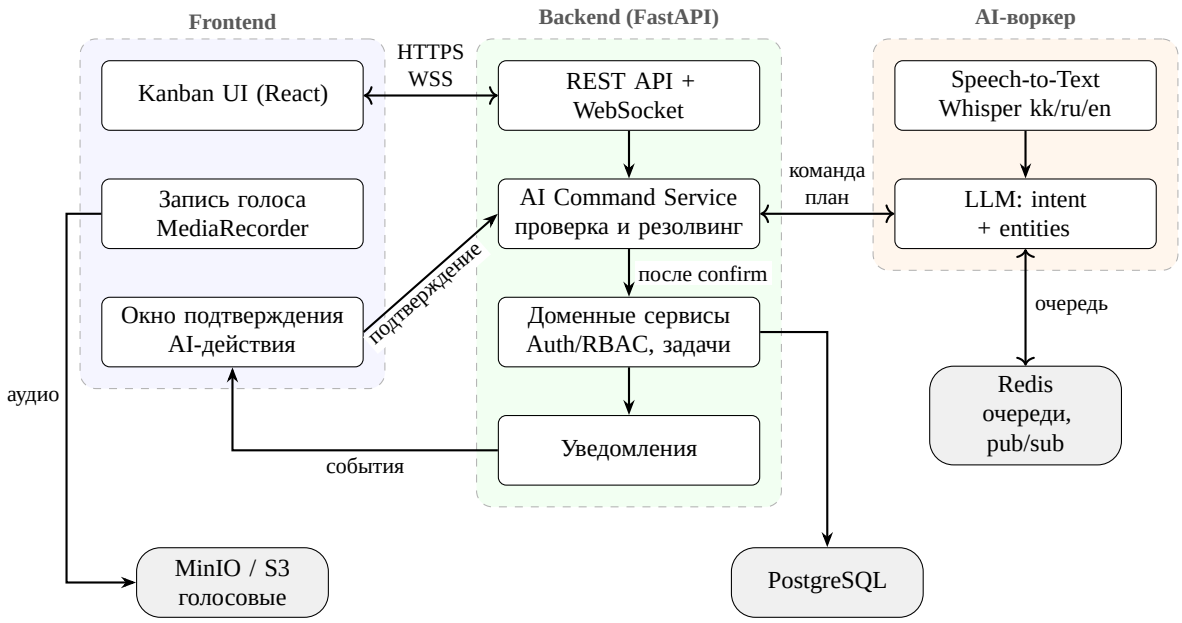


Рис. 1: Архитектура компонентов системы. AI-воркер читает аудио из MinIO и возвращает план действий; данные меняет только backend после подтверждения.

3.1. AI-конвейер

Voice Input → Speech-to-Text → Intent Recognition → Entity Extraction →
Validation & Resolution → User Confirmation → Backend Action → Kanban Update

Шаг	Где	Технология
Voice Input	Браузер	MediaRecorder API (WebM/Opus), загрузка в MinIO через backend
Speech-to-Text	AI-воркер	Whisper large-v3-turbo + Whisper, дообученный на казахском (раздел 4)
Intent Recognition + Entity Extraction	AI-воркер	LLM с tool calling: один вызов возвращает список действий с параметрами (раздел 5)
Validation & Resolution	Backend, без LLM	Pydantic-схемы, поиск исполнителя среди участников проекта, поиск задач (pg_trgm), проверка дат и прав
User Confirmation	Frontend	Карточки-превью с подсвеченными пропущенными и неоднозначными полями; редактирование транскрипта и полей
Backend Action	Backend	Те же доменные сервисы, что и для ручных действий в UI (единая проверка прав и аудит)
Kanban Update	Frontend	WebSocket-событие, доска обновляется у всех участников

Ключевой принцип. LLM только *понимает команду и предлагает* действие. Всё, что меняет данные, выполняет обычный backend-код после подтверждения пользователя, с теми же проверками прав, что и для ручных действий. Поэтому AI не может изменить чужую задачу или создать задачу без подтверждения.

4. Speech-to-Text

4.1. Сравнение

Критерий	Whisper large-v3-turbo (self-hosted)	Whisper, дообученный на KSC2 (self-hosted)	ElevenLabs Scribe (облако)	OpenAI gpt-4o-transcribe (облако)
Казахский	Слабо: путает с турецким, в исследовании WER ≈77%	**Хорошо: WER ≈15–18%** на тесте KSC2	Хорошо (заявлено WER 3,1% на FLEURS)	Средне–хорошо
Русский / английский	**Отлично**	Хуже базового	Отлично	Отлично
Смешанная речь	Автоопределение	Частично	Автоопределение	Автоопределение
Скорость	≈1–2 с на 10 с аудио (GPU)	Аналогично	1–3 с + сеть	1–3 с + сеть
Стоимость	Только железо	Только железо	≈$0,22 / час	$0,006 / мин
Приватность	**Голос не покидает сервер**	**Голос не покидает сервер**	Данные у провайдера	Данные у провайдера
Масштабирование	GPU-воркеры	GPU-воркеры	Автоматически	Автоматически

4.2. Решение: self-hosted STT с маршрутизацией по языку и облачный fallback

1. **Автоматическое определение языка.** Whisper large-v3-turbo (`faster-whisper`) вычисляет вероятности языков; они ограничиваются тремя языками (kk, ru, en) и нормируются.
2. **Маршрутизация без подсказки.** Казахская модель используется, если казахский — самый вероятный язык *или* $p(\text{kk}) \geq 0{,}30$. Порог нужен для смешанной речи: казахская фраза с русскими и английскими терминами часто определяется как русская, но с заметной долей казахского. В остальных случаях русскую и английскую речь распознаёт Whisper large-v3-turbo с определённым языком.
3. **Язык, выбранный пользователем.** Если пользователь выбрал язык в интерфейсе, определение не выполняется: kk — казахская модель, ru/en — Whisper large-v3-turbo. Для смешанной речи интерфейс предлагает казахский.
4. Пользователь может **исправить транскрипт** перед обработкой.
5. **Fallback:** если локальный STT недоступен или перегружен, используется облачный провайдер (включается в настройках workspace).

Порог $0{,}30$ — начальное значение; он задаётся параметром и подбирается по бенчмарку. Бенчмарк (`benchmarks/stt_benchmark.py`) оценивает многоязычные модели в двух режимах отдельно: **auto** (автоматическое определение языка — основной сценарий продукта) и **hint** (язык выбран пользователем). Для режима auto дополнительно считается точность определения языка и то, куда маршрутизируются смешанные записи. Финальная казахская модель выбирается по результатам бенчмарка.

Обоснование. Голос — самые чувствительные данные системы. Self-hosted STT решает вопрос приватности и стоимости при росте числа пользователей, а дообученная модель закрывает слабость Whisper на казахском. Облачный fallback повышает надёжность (R8).

5. LLM для распознавания команд

5.1. Что должна уметь модель

Из одной фразы, часто с ошибками распознавания и смешанными языками, получить **план действий** строго по схеме:

```
{
  "actions": [
    {
      "type": "create_task",
      "params": {
        "title": "Подготовить презентацию проекта",
        "assignee_mention": "Данияр",
        "deadline": "2026-10-23",
        "deadline_phrase": "до пятницы",
        "priority": "high"
      },
      "missing": [],
      "confidence": 0.92
    }
  ],
  "clarification_needed": null
}
```

Требования: tool calling или JSON Schema, понимание kk/ru/en, разбор нескольких поручений в одной фразе, казахские падежные формы имён («Данияpға» → «Данияр»), относительные даты.

5.2. Сравнение

Критерий	Локальная LLM 7–8B (Qwen 3 / ISSAI KazLLM, Ollama)	Облачная LLM Flash/mini-класса (Gemini Flash, OpenAI mini)
Казахский	Qwen — средне; KazLLM — хорошо, но слабее в tool calling	Хорошо
Tool calling, несколько действий в одной фразе	Нестабильно на 7–8B, особенно с kk	**Надёжно**
Скорость	3–15 с без мощной GPU	**1–3 с**
Стоимость	Только железо	Оценка: ≈1,5 тыс. входных + 300 выходных токенов на команду ≈ **$0,005 за команду**
Масштабирование	Нужны дополнительные GPU	Автоматически
Приватность	Полная	В LLM уходит только текст команды и имена участников проекта; без голоса и содержимого задач

5.3. Решение: гибрид через единый интерфейс `CommandParser`

- **Основной режим — облачная LLM Flash/mini-класса** с tool calling. Для реального продукта решают качество разбора сложных команд на казахском, скорость ответа (пользователь ждёт превью карточки) и масштабирование без закупки GPU.

- **Приватный (on-premise) режим — локальная LLM через Ollama** (Qwen 3 или KazLLM 8B). Включается конфигурацией для организаций, которым нельзя отправлять текст во внешние сервисы, и служит fallback при недоступности облачного API.
- Оба провайдера реализуют один интерфейс и одну JSON-схему. Конкретная модель выбирается по тесту на наших 30+ сценариях: точность определения намерения, точность полей, доля выдуманных значений, время ответа.

Почему изменено решение версии 2 (полностью локально). Без мощной GPU локальная модель 7–8B отвечает слишком медленно и нестабильно разбирает несколько действий в одной казахской фразе. Это противоречит критериям «скорость» и «качество интеграции AI». Голос при этом обрабатывается только на нашем сервере, а локальный режим LLM сохранён как опция.

5.4. Защита от выдуманных данных

Риск	Механизм
Выдуманный исполнитель	LLM возвращает только упоминание (`assignee_mention`). Backend ищет его среди участников проекта: нормализация казахских падежей, транслитерация, нечёткий поиск. Не найден или найдено несколько — поле помечается, пользователь выбирает из списка
Выдуманный срок	LLM возвращает дату и исходную фразу (`deadline_phrase`). Если фразы нет в транскрипте, дата отбрасывается. Backend проверяет дату: не в прошлом, с учётом часового пояса пользователя
Неверная задача при изменении	LLM возвращает описание задачи. Backend ищет кандидатов (pg_trgm по названию в доступных проектах): 0 — «не найдено», больше 1 — уточнение у пользователя
Выдуманные задачи в ответах (Search, Summary)	Запрос выполняет backend по реальным данным. LLM формирует только фильтр (статус, исполнитель, срок), в ответ подставляются реальные строки из БД
Действие без подтверждения	Изменения выполняются только через `POST /ai-commands/{id}/confirm`

6. Frontend

Критерий	React + TypeScript (Vite)	Vue 3	Angular
Drag-and-drop для Kanban	dnd-kit, много готовых компонентов	vuedraggable	CDK Drag&Drop
Серверное состояние и real-time	TanStack Query + WebSocket	Pinia + composables	RxJS
Типизация с backend	Генерация типов из OpenAPI	Аналогично	Аналогично
Порог входа и рынок	Самый распространённый	Простой	Высокий

Решение: React + TypeScript + Vite. Tailwind CSS и shadcn/ui для компонентов, dnd-kit для перетаскивания карточек, TanStack Query для данных, клиент API генерируется из OpenAPI-спецификации FastAPI (единые типы на фронте и бэке), react-i18next для kk/ru/en. Голос записывается через браузерный MediaRecorder API, отдельное мобильное приложение не нужно.

7. Backend

Критерий	FastAPI	Django + DRF	Flask
Асинхронность (AI-вызовы, WebSocket)	Нативная	Частичная	Через расширения
Валидация	Pydantic: одни и те же схемы для AI-действий, API и проверки	Serializers	Вручную
OpenAPI-спецификация	Автоматически	Через дополнения	Через дополнения
WebSocket	Встроено	Django Channels	Расширения

Решение: Python 3.12 + FastAPI, SQLAlchemy 2.0 (async) и Alembic. Модульная структура по доменам: `auth`, `workspaces`, `projects`, `tasks`, `comments`, `notifications`, `ai_commands`, `audit`.

AI-воркер — отдельный процесс (ARQ или Celery + Redis). Модели STT загружаются в память один раз, API не блокируется, воркеры масштабируются отдельно.

8. База данных и хранилища

Критерий	PostgreSQL	MongoDB
Модель данных (workspace → проект → доска → колонка → задача, участники, роли)	Реляционная, внешние ключи, транзакции	Связи вручную
Права доступа и целостность	Ограничения на уровне БД	В коде
Поиск задач по описанию из голоса	pg_trgm; при необходимости pgvector	Atlas Search
Гибкие данные AI (транскрипт, план)	JSONB	Нативно
Аудит и история	Append-only таблица	Коллекция

Решение:

- **PostgreSQL 16** — основная БД (pg_trgm, опционально pgvector).
- **Redis** — очередь AI-задач, pub/sub для WebSocket между инстансами API, rate limiting, ключи идемпотентности.
- **MinIO** (S3-совместимое хранилище) — голосовые сообщения; в облаке заменяется на S3 без изменения кода.

Ключевые сущности (подробно — в Database Schema): `users`, `workspaces`, `workspace_members`, `projects`, `project_members`, `boards`, `columns`, `tasks`, `task_comments`, `task_history`, `notifications`, `voice_messages`, `ai_commands`, `audit_log`.

9. Безопасность и защита данных

Требование	Решение
Аутентификация	JWT: короткий access-токен и refresh в httpOnly cookie; пароли хешируются Argon2
Права доступа	RBAC: роли workspace (owner, admin, member) и проекта (manager, member, viewer); проверка в каждом сервисе, включая AI-действия
Голосовые сообщения	Приватный bucket, шифрование at rest, доступ по короткоживущим presigned URL, автоудаление через 30 дней (настраивается); транскрипт сохраняется в истории
Данные в облачной LLM	Только текст команды и имена участников проекта; без голоса, email и содержимого других задач. Для on-premise — локальная LLM
Повторное создание задач	Статус AI-команды `pending` → `confirmed` → `executed`; подтверждение идемпотентно, повторный клик ничего не дублирует; предупреждение о похожей недавно созданной задаче
Недоступность AI	Таймауты и повторы, переключение на fallback-провайдера, понятное сообщение; ручное создание задачи всегда доступно
История действий	`task_history` (что, кем, когда, через UI или AI) и `audit_log`
Транспорт	HTTPS/WSS, CORS, rate limiting на AI-эндпоинты

10. Уведомления

- **In-app в реальном времени:** WebSocket — назначение, изменение статуса, комментарий, упоминание.
- **Email:** SMTP (при разработке — Mailpit) о назначении и приближающихся сроках.
- **Напоминания о сроках:** периодическая задача (ARQ cron или Celery beat) раз в 15 минут находит задачи со сроком в ближайшие 24 часа и просроченные.
- Пользователь сам настраивает, какие уведомления получать.

11. Итоговый стек

Слой	Технология	Ключевая причина
Языки	kk, ru, en, смешанная речь	Требование продукта
Frontend	React + TypeScript + Vite, Tailwind + shadcn/ui, dnd-kit, TanStack Query, react-i18next	Зрелая экосистема для Kanban, типы из OpenAPI
Запись голоса	MediaRecorder API (браузер)	Без отдельного приложения
Backend	Python 3.12 + FastAPI, SQLAlchemy 2.0, Alembic	Async, Pydantic, автоматический OpenAPI
AI-воркер	ARQ / Celery + Redis	Не блокирует API, масштабируется отдельно
Speech-to-Text	Self-hosted: Whisper large-v3-turbo (ru/en, определение языка) + Whisper, дообученный на KSC2 (kk); облачный fallback опционально	Голос не покидает сервер; качество казахского
LLM	Облачная LLM Flash/mini-класса с tool calling; локальная Qwen 3 / KazLLM (Ollama) — on-premise режим и fallback	Скорость и надёжность; приватный режим сохранён

Слой	Технология	Ключевая причина
БД	PostgreSQL 16 + pg_trgm (+ pgvector)	Реляционная модель, поиск задач, JSONB
Кэш, очереди, pub/sub	Redis	Очереди AI, WebSocket между инстансами, идемпотентность
Голосовые	MinIO (S3), шифрование, TTL 30 дней	Приватность, переносимость в облако
API	REST (OpenAPI) + WebSocket	CRUD и real-time доска
Auth	JWT + refresh, Argon2, RBAC	Права на workspace и проект
Уведомления	WebSocket + email, периодические напоминания	Изменения и сроки
Инфраструктура	Docker Compose (frontend, api, worker, postgres, redis, minio, ollama)	Запуск одной командой
Тестирование	pytest, httpx, Playwright (e2e), eval-набор AI-сценариев (30+), jiwer (WER)	Проверка и AI, и операций системы
CI и процесс	GitLab CI (линтеры, тесты, сборка), YouTrack	Цифровой след

12. Проверка выбора

Что проверяем	Метрика	Набор
STT	WER / CER по языкам отдельно для режимов auto и hint; время	20+ записей kk/ru/en/mixed, `stt_benchmark.py`
Определение языка	Точность определения kk/ru/en (режим auto); маршрутизация смешанных записей	Те же записи
Распознавание намерения	Accuracy по 8 действиям и классу «не команда»	30+ сценариев из Test Cases
Извлечение полей	Точность по полям (title, assignee, deadline, priority)	Те же сценарии
Выдуманные данные	Доля значений, которых нет в транскрипте	Те же сценарии
Скорость	Время от конца записи до превью (цель — менее 5 с)	Те же сценарии
Сравнение LLM	Облачная и локальная на одном наборе	Подтверждение раздела 5

Результаты сохраняются в `benchmarks/results/` и сводятся в этот документ.

13. Риски

Риск	Митигация
Низкая точность STT на смешанной речи	Ручной выбор языка, редактирование транскрипта до обработки
Недоступность облачной LLM	Fallback на локальную модель; ручное создание задач всегда работает
Ошибочное сопоставление исполнителя или задачи	Детерминированный поиск и подтверждение; уточнение при нескольких совпадениях
Нет GPU для self-hosted STT	Модели меньшего размера и int8 на CPU; облачный STT-fallback

Риск	Митигация
Утечка голосовых данных	Приватное хранилище, шифрование, TTL, доступ по presigned URL

14. Источники

[1] Исследование STT/TTS для казахского языка (Whisper, GPT-4 Transcribe, ElevenLabs, Soyle). https://api.crossref.org/works/10.3390%2FINFO16100879

[2] Whisper, дообученный на KSC2: https://huggingface.co/akuzdeuov/whisper-base.kk, https://huggingface.co/abilmansplus/whisper-turbo-ksc2

[3] Whisper large-v3 для казахского (tulpar). https://huggingface.co/olzhasAl/whisper-large-v3-tulpar

[4] faster-whisper large-v3-turbo. https://huggingface.co/deepdml/faster-whisper-large-v3-turbo-ct2

[5] ISSAI Soyle. https://github.com/IS2AI/Soyle

[6] ISSAI KAZ-LLM. https://issai.nu.edu.kz/kazllm/

[7] ElevenLabs Scribe, казахский. https://elevenlabs.io/speech-to-text/kazakh

[8] ElevenLabs API pricing. https://developer.puter.com/tutorials/elevenlabs-api-pricing/

[9] OpenAI API pricing. https://developers.openai.com/api/docs/pricing

[10] Gemini Flash: structured output, tool calling, цены. https://docs.apiyi.com/en/api-capabilities/gemini-3-6-flash/overview.md